

MANUAL HYPERPARAMETER SELECTION FORM

2D CNN Model — Dogs vs. Cats Image Classification

Name: Arnold B. Platon

Program: Doctor in Information Technology

Use this form to record, justify, and approve each hyperparameter value chosen by hand before a training run. The "Notebook Default" column shows the value currently used in `Loading_Dataset_and_CNN.ipynb` for reference — enter the value actually used for this run under "Chosen Value", even if it matches the default.

1. Project / Experiment Information

Project / Experiment Title	Hypertuning using Keras
Model Architecture	2D Convolutional Neural Network (Conv2D)
Dataset Used	Kaggle Dogs vs. Cats images (Dog/ and Cat/ folders)
Notebook / File Reference	Loading_Dataset_and_CNN.ipynb
Prepared By	Arnold B. Platon
Date	09/19/2026
Experiment / Trial No.	

2. Data Preprocessing Parameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification
Image Size (IMG_SIZE, square)	e.g. 50 / 80 / 100 / 128 / 150	100	64	To make image processing faster.
Color Mode	Grayscale / RGB	Grayscale	Grayscale	To make image processing faster.
Pixel Normalization	X/255.0 / none	X/255.0	X/255.0	
Shuffle Training Data	Yes / No	Yes (random.shuffle)	Yes (Random Shuffle)	To present the images of dogs and cats randomly especially during the data split
Held-out Test Set (separate from training)	Yes / No	No (validation_split only)	No (validation_split only)	None
Validation Split (of training set)	0.0 - 0.3	0.1	0.15	15% is a good value to held out the test data for testing purposes.

3. Model Architecture Hyperparameters

Trial 1

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Conv2D Layer 1 – Filters	16 / 32 / 64 / 128	14	16 to 64 step 16	Variably changing the values during training	84.28%	81.93%
Conv2D Layer 1 – Kernel Size	(3,3) / (5,5)	(3,3)	(3,3)			
Conv2D Layer 1 – Activation	relu / tanh / elu / leaky_relu	relu	relu			
Pooling Layer 1 – Type & Pool Size	MaxPooling2D / AveragePooling2D; (2,2) / (3,3)	MaxPooling2D, (2,2)	MaxPooling2D, (2,2)			
Dense (Fully Connected) Units	16 / 32 / 64 / 128	64	16 to 64 Step 16	Variably changing the values during training		
Dropout Rate	0.0 - 0.5	None (not used)	None			
Output Layer Activation	sigmoid (binary) / softmax (multi-class)	sigmoid	sigmoid			

4. Training Hyperparameters

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Loss Function	binary_crossentropy / categorical_crossentropy	binary_crossentropy	Binary cross entropy		84.28%	81.93%
Optimizer	adam / sgd / rmsprop	adam	Adam			

Hyperparameter	Options / Range Considered	Notebook Default	Chosen Value	Justification	Training Accuracy	Validation Accuracy
Learning Rate	0.1 / 0.01 / 0.001 / 0.0001	0.001 (Adam default)	hp.Choice ("lr", values=[1e-2, 1e-3, 1e-4]) based on the tuned model, the 1e-3 or 0.001 won	Variably changing the values during training		
Batch Size	8 / 16 / 32 / 64 / 128	32	32	I did not set a batch size parameter, so Keras uses the default 32		
Number of Epochs	e.g. 10 - 200	200	20	20 epochs allowed as maximum		
Early Stopping	Yes (patience = ____) / No	No	Yes Patience= 4	To allow only 4 epochs once the val_loss is starting to climb to higher value		

5. Evaluation Results (after training with chosen values) - Final Model

Metric	Value Obtained	Target / Acceptable Range
Training Accuracy	0.8428	
Validation Accuracy	0.8193	
Test Accuracy	0.8130	
Test Loss	0.4056	
Precision / Recall / F1 (macro avg)	0.8130	

5. a. Paste here all screenshots/figures of the tuned model

Training many versions of your model with different hyperparameter combinations and tracking which one performs best.

Trial 30 Complete [00h 11m 29s]
val_accuracy: 0.5018820762634277

Best val_accuracy So Far: 0.8262233138084412
Total elapsed time: 02h 53m 18s

Get the best hyperparameter combination (not necessarily the best trained model object), and retrain a fresh model with those settings from scratch for a proper full run.

Epoch 1/20

565/565 ————— **83s** 144ms/step - accuracy: 0.6118 - loss: 0.6439 - val_accuracy: 0.7048 - val_loss: 0.5742

Epoch 2/20

565/565 ————— **85s** 149ms/step - accuracy: 0.7229 - loss: 0.5424 - val_accuracy: 0.7582 - val_loss: 0.5091

Epoch 3/20

565/565 ————— **82s** 145ms/step - accuracy: 0.7689 - loss: 0.4819 - val_accuracy: 0.7858 - val_loss: 0.4659

Epoch 4/20

565/565 ————— **145s** 151ms/step - accuracy: 0.7876 - loss: 0.4489 - val_accuracy: 0.7883 - val_loss: 0.4673

Epoch 5/20

565/565 ————— **82s** 146ms/step - accuracy: 0.8086 - loss: 0.4142 - val_accuracy: 0.7958 - val_loss: 0.4459

Epoch 6/20

565/565 ————— **81s** 143ms/step - accuracy: 0.8232 - loss: 0.3856 - val_accuracy: 0.8043 - val_loss: 0.4271

Epoch 7/20

565/565 ————— **83s** 145ms/step - accuracy: 0.8428 - loss: 0.3522 - val_accuracy: 0.8193 - val_loss: 0.4032

Epoch 8/20

565/565 ————— **95s** 168ms/step - accuracy: 0.8545 - loss: 0.3277 - val_accuracy: 0.8225 - val_loss: 0.4145

Epoch 9/20

565/565 ————— **131s** 149ms/step - accuracy: 0.8723 - loss: 0.2955 - val_accuracy: 0.8221 - val_loss: 0.4194

Epoch 10/20

565/565 ————— **83s** 147ms/step - accuracy: 0.8871 - loss: 0.2669 - val_accuracy: 0.8240 - val_loss: 0.4301

Epoch 11/20

565/565 ————— **140s** 143ms/step - accuracy: 0.8988 - loss: 0.2384 - val_accuracy: 0.8215 - val_loss: 0.4389

118/118 ————— **4s** 38ms/step - accuracy: 0.8131 - loss: 0.4056

Test Accuracy: 0.8130666613578796

Test Loss: 0.40562134981155396

118/118 ————— **6s** 46ms/step

Precision (macro avg): 0.8130924042538181

Recall (macro avg): 0.8130666666666666

F1-score (macro avg): 0.8130628249000104

6. Overall Justification / Remarks

Summarize why this combination of hyperparameters was chosen (e.g., prior trial results, literature basis, computational constraints) and any observations (overfitting, underfitting, convergence issues).

Manual hyperparameter tuning relies on a person testing one combination at a time based on intuition, which is slow and cannot realistically cover more than a handful of the many possible combinations. Hyperparameter tuning with Keras, through KerasTuner's Hyperband search, automates this process by systematically testing many configurations and cutting off weak ones early to save computation (Li et al., 2018). In this project, that automation ran roughly 26 trials and identified a better-performing architecture than would likely have been found through manual trial and error alone. Automated tuning also kept a complete, reproducible record of every trial, which proved essential when the search results needed to be recovered after a Colab disconnect. However, this project showed that automation does not protect against upstream mistakes, since a data preprocessing bug caused every trial to fail identically until the pipeline itself was corrected. Overall, Keras-based tuning is faster, more thorough, and more scalable than manual tuning, but it still depends on the person verifying that the underlying data and code are correct before trusting its results.